

ShadowDrive++ — Complete 12-Week Master Plan

(Team: Rohan & Shabd)

A full, production-grade, MIT/Stanford-level distributed systems project roadmap.

This document contains: - Full integrated **Learn → Implement → Test** sequencing - Clear **team split** between **Rohan** (Client Agent Owner) and **Shabd** (Backend Owner) - What to learn, in what order, with curated resources - When to implement each module - How to integrate both halves into a unified system - Weekly milestones for both members - Final merging + debugging strategy

PROJECT OVERVIEW

ShadowDrive++ is a minimal Dropbox-like **distributed file sync system** with: - File watching - Hashing + diff detection - Metadata database - Client-server sync protocol - Storage backend via S3/MinIO - Conflict resolution - Background workers - Version history dashboard

You (Rohan) will own everything that happens on a user's machine. Shabd will own everything that happens on the server. Together, you build a professional distributed system.

HIGH-LEVEL DIVISION OF WORK

Rohan — Client Agent + Sync Logic Owner (50%)

You build: - File watching system - Hashing + diff detection - Local metadata shadow DB - Upload worker - Download worker - Client-side conflict detection - Client state reconciliation

Your learning focus: - OS file systems - Hashing algorithms - Concurrency (threads, async) - State machines - Networking basics

Shabd — Backend + DBMS + Storage Owner (50%)

He builds: - PostgreSQL schema - FastAPI/gRPC backend - Metadata processing - File chunk upload/download endpoints - MinIO storage layer - Server-side conflict resolution - Redis background workers - Dashboard UI

His learning focus: - DBMS internals - Storage systems - API design - Distributed metadata management - Background job processing

CURATED RESOURCES (Rohan + Shabd)

File Systems & Watchers (Rohan)

- Python `watchdog` docs
- DigitalOcean: inotify explained
- Linux journey: filesystem basics

Hashing & Diff Algorithms (Rohan)

- Python `hashlib` docs
- Rsync algorithm: "Rsync Rolling Checksum" article
- Karp–Rabin rolling hash (cp-algorithms)

PostgreSQL + Schema Design (Shabd)

- postgresqltutorial.com
- UseTheIndexLuke.com (indexing)
- SQLAlchemy docs

FastAPI Backend (Shabd)

- FastAPI documentation
- freeCodeCamp FastAPI video

gRPC (optional, both)

- gRPC docs
- freeCodeCamp grpc python tutorial

MinIO Storage (Shabd)

- MinIO official documentation
- Python boto3 MinIO videos

Redis Queues (both optional)

- Redis Queue (RQ) docs

System Design Basics (both)

- ByteByteGo: free YT content
-



THE 12-WEEK INTEGRATED LEARN-IMPLEMENT PLAN

Each week includes: 1. **What to learn** 2. **Who learns it (Rohan or Shabd)** 3. **What to implement immediately** 4. **Deliverables**

No ambiguity, no ifs or buts — do the steps in order.



PHASE 1 — FOUNDATIONS + FIRST IMPLEMENTATIONS (WEEK 1-4)

WEEK 1 — File Watching + Filesystem Foundations

Rohan Learns:

- OS-level file events
- Watchdog library
- File types, inodes, timestamps

Rohan Implements:

- A folder watcher script
- Logs every create/modify/delete
- Stores events in a local JSON/SQLite file

Deliverable (Rohan):

`watcher.py` + log output proving events captured

Shabd Learns:

- Basics of DBMS schema thinking
- SQL relationships

Shabd Implements:

- Initial ER diagram draft
-

WEEK 2 — Hashing + Diffing + Local State

Rohan Learns:

- SHA256, MD5
- Chunking strategy (4 MB)
- Rolling hash (optional)

Rohan Implements:

- Compute hash for each file
- Compare new vs old hash
- Detect: new/update/delete
- Maintain local shadow state in SQLite

Deliverable (Rohan):

`diff_engine.py` that outputs detected changes

Shabd Learns:

- PostgreSQL basics
- Transactions, isolation

Shabd Implements:

- DB schema:
- files
- versions
- devices
- file_device_map

Deliverable (Shabd):

SQL schema file + migrations

WEEK 3 — Networking + API Foundations

Rohan Learns:

- HTTP basics
- REST semantics
- Sending file chunks via HTTP

Rohan Implements:

- Small client script to POST a file to server

Shabd Learns:

- FastAPI routing
- Pydantic models
- Handling file uploads

Shabd Implements:

- `/register_device`
- `/upload_metadata`
- `/get_metadata`

Deliverables:

- Working server skeleton
 - Client talks to server successfully
-

WEEK 4 — Sync Protocol (DONE TOGETHER)

Both Learn:

- Sync cycles
- Idempotency
- Version numbering
- Conflict rules

Both Implement:

- A written protocol document containing:
 - metadata JSON format
 - API contract
 - version rules
 - chunk rules
 - frequency of sync

Deliverable:

ShadowDrive++ Sync Protocol v1.0.pdf



PHASE 2 — FULL FEATURE DEVELOPMENT (WEEK 5-9)

WEEK 5 — Upload Pipeline

Rohan Learns:

- Threading basics
- Background workers

Rohan Implements:

- Upload worker thread
- Upload file chunks sequentially
- Update metadata accordingly

Shabd Learns:

- MinIO bucket operations

Shabd Implements:

- `/upload_chunk`
- Chunk storage mapping

Integration Deliverable:

Rohan's agent uploads a file → metadata & chunks appear in server.

WEEK 6 — Download Pipeline

Rohan Implements:

- Compare local metadata with server's
- Download missing chunks
- Reconstruct files locally

Shabd Implements:

- `/download_chunk`
- Metadata diff endpoint

Deliverable:

Full local → server → local roundtrip works.

WEEK 7 — Conflict Resolution

Rohan Learns:

- State machines for sync
- Local conflict markers

Rohan Implements:

- Client detects simultaneous edits

Shabd Learns:

- LWW
- Vector clocks (optional)

Shabd Implements:

- Server-side conflict resolution rules

Deliverable:

Conflicting edits produce conflict files.

WEEK 8 — Reliability (Queues + Retries)

Rohan Learns:

- Retry strategies
- Exponential backoff

Rohan Implements:

- Retry queue for failed uploads

Shabd Learns:

- Redis Queue

Shabd Implements:

- Background worker to reprocess failed requests

Deliverable:

System tolerates network failures.

WEEK 9 — Dashboard

Shabd Implements:

- Streamlit/React UI showing:
 - file list
 - version list
 - conflict warnings
 - device activity

Rohan Supports:

- Provides metadata formats for UI

Deliverable:

A visual interface of ShadowDrive++.



PHASE 3 — FINALIZATION (WEEK 10–12)

WEEK 10 — Dockerization

Both Learn:

- Dockerfile basics
- Docker Compose

Both Implement:

- Containers for:
 - Backend
 - PostgreSQL
 - MinIO
 - Redis

Deliverable:

`docker-compose up` launches full system.

WEEK 11 — End-to-End Testing + Load Testing

Both Implement:

- Multi-device sync tests
- Conflict scenario tests
- Network failure simulations

Deliverable:

Test report documenting all scenarios.

WEEK 12 — Documentation + Demo

Both Implement:

- README with diagrams
 - Final architecture diagram
 - Sequence diagrams
 - Demo video (2–3 min)
-



FINAL INTEGRATION STRATEGY

Week 4 → Protocol freeze

Ensures both codebases stay compatible.

Week 5–9 → API-driven integration

Each new feature is tested end-to-end.

Week 10 → Docker integration

Everything runs together.

Week 11 → Final debugging

Fix inconsistencies, metadata mismatches, race conditions.

Week 12 → Publish

Show polished project with confidence.

FINAL OUTCOME

After 12 weeks: - Rohan becomes strong in OS, concurrency, hashing, sync logic. - Shabd becomes strong in DBMS, backend, storage, system design. - Together: you build a real distributed system. - Perfect alignment with next semester's ADDA, DBMS, TOC. - Resume gains a **FAANG-level** systems project.

If you want: I can now add **architecture diagrams, DB schema diagrams, API definitions, or GitHub repo structure** into this document.